

Trois méthodes de déploiement

Dans les modules 1 et 2, vous avez appris à exécuter des agents avec `adk web`, l'interface Web visuelle pour le développement et les tests. Vous avez par exemple utilisé la commande suivante :

```
adk web
```

Cette commande ouvre une UI basée sur un navigateur, dans laquelle vous pouvez discuter de manière interactive avec votre agent.

Mais `adk web` n'est pas la seule façon d'exécuter des agents. Dans ce module, vous allez découvrir trois autres méthodes :

- `adk run` : pour une exécution basée sur le terminal
- `adk api_server` : pour le déploiement en tant que service d'API
- **Exécution programmatique** : pour intégrer des agents dans des applications Python

Méthode 1 : Exécution depuis le terminal avec `adk run`

Référence : [Documentation ADK – Exécuter votre agent](#)

Description

`adk run` vous permet d'interagir avec votre agent directement depuis le terminal, sans ouvrir de navigateur Web.

Comment l'utiliser ?

Depuis votre répertoire d'agent :

```
# Accéder à votre répertoire de projet d'agent
cd my_first_agent

# Exécuter l'agent
adk run
```

Comportement attendu :

- Le terminal devient interactif.
- Saisissez votre message et appuyez sur **Entrée**.
- L'agent répond dans le terminal.
- Saisissez un autre message pour poursuivre la conversation.
- Appuyez sur `Ctrl+C` pour quitter.

Depuis le répertoire parent :

Si vous vous trouvez dans le répertoire parent (par exemple, adk-workspace)
adk run my_first_agent

Exemple d'interaction

```
$ adk run my_first_agent
```

Vous : Comment **puis-je** résoudre l'équation $x + 5 = 10$?

Agent : Excellente question ! Essayons **de** trouver la solution à ce problème ensemble.
À **votre** avis, que
devons-nous faire **pour** isoler l'inconnue x d'un côté de l'équation ?

Vous : Soustraire **5** des deux côtés ?

Agent : Exactement ! Il **s'agit** de la bonne approche. Quand on soustrait **5** des deux côtés...

Quand utiliser **adk run**

Convient aux cas d'utilisation suivants :

-  Tests rapides pendant le développement
-  Workflows en ligne de commande
-  Environnements de serveur sans IUG
-  Scripts de test automatisés
-  Pipelines CI/CD

Pas idéal pour les cas d'utilisation suivants :

-  Présentation aux personnes concernées (utiliser **adk web** à la place)
-  Débogage de conversations complexes (utiliser **adk web** à la place)

Méthode 2 : Serveur d'API avec **adk api_server**

Référence : [Documentation ADK – Configuration d'agent : Exécuter votre agent](#)

Description

adk api_server exécute votre agent en tant que service d'API REST, ce qui permet à d'autres applications d'envoyer des requêtes à votre agent via HTTP.

Comment l'utiliser ?

Démarrez le serveur d'API :

```
# Depuis votre répertoire d'agent  
cd my_first_agent  
adk api_server
```

Ou depuis le répertoire parent :

```
adk api_server my_first_agent
```

Résultat attendu :

```
INFO:      Started server process [12345]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Effectuer des tests avec cURL

Une fois le serveur en cours d'exécution, vous pouvez envoyer des requêtes à l'aide de **curl** :

```
# Dans une fenêtre de terminal distincte
curl -X POST http://localhost:8000/your-endpoint \
  -H "Content-Type: application/json" \
  -d '{"message": "Quelle est la valeur de x dans l'équation 2x + 5 = 13 ?"}'
```

Remarque : Pour en savoir plus sur l'utilisation de l'API, consultez la [documentation sur les tests ADK](#).

Quand utiliser **adk api_server**

Convient aux cas d'utilisation suivants :

-  Intégration d'agents dans des applications Web
-  Backends d'applications mobiles
-  Architectures de microservices
-  Tests en préproduction
-  Développement d'API en local avant le déploiement sur Cloud Run

Pas idéal pour les cas d'utilisation suivants :

-  Développement interactif (utiliser **adk web** à la place)
-  Tests rapides (utiliser **adk run** à la place)

Méthode 3 : Exécution programmatique avec Python

Référence : [Documentation ADK – Tutoriel sur l'équipe d'agents](#)

Description

Vous pouvez exécuter des agents directement dans votre code Python, ce qui vous offre un contrôle programmatique total. Cette méthode est utile dans les cas suivants :

- Création d'applications personnalisées avec des agents
- Notebooks Jupyter/Google Colab
- Pipelines de traitement des données
- Intégrations personnalisées

Exemple complet (prêt à être copié-collé)

Cet exemple autonome est prêt à être exécuté dans un script Python ou un notebook Jupyter :

```
"""
Exemple complet : Exécuter un agent ADK de manière programmatique
Copiez l'intégralité de ce bloc de code pour l'exécuter dans un script ou un notebook
Python.
"""

# Étape 1 : Installez ADK (exécutez cette commande dans un terminal ou une cellule de
notebook)
# pip install google-adk

# Étape 2 : Définissez votre clé API
# Option A : Définissez-la comme variable d'environnement avant l'exécution
# export GOOGLE_API_KEY=your-api-key-here
# Option B : Annulez la mise en commentaire et utilisez ce code :
# import os
# os.environ['GOOGLE_API_KEY'] = 'your-api-key-here'
# os.environ['GOOGLE_GENAI_USE_VERTEXAI'] = 'FALSE'

# Étape 3 : Importez les bibliothèques requises
import asyncio
from google.adk.agents.llm_agent import Agent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai.types import Content, Part

# Étape 4 : Définissez votre agent
agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor',
    instruction="""Tu es un professeur de mathématiques patient.
Guide les élèves dans la résolution de problèmes étape par étape.
Ne te contente pas de donner des réponses, aide-les à trouver des solutions."""
)

# Étape 5 : Configurez la session et l'exécuteur
APP_NAME = "math_tutor_app"
USER_ID = "student_1"
SESSION_ID = "session_001"
```

```

session_service = InMemorySessionService()
runner = Runner(
    agent=agent,
    app_name=APP_NAME,
    session_service=session_service
)

# Étape 6 : Définissez la fonction async pour exécuter l'agent
async def run_agent():
    # Créer une session
    session = await session_service.create_session(
        app_name=APP_NAME,
        user_id=USER_ID,
        session_id=SESSION_ID
    )
    print(f"Session created: {SESSION_ID}\n")

    # Préparer le message utilisateur
    user_message = Content(
        role="user",
        parts=[Part(text="Comment résoudre l'équation  $2x + 5 = 13$  ?")]
    )

    # Exécuter l'agent et collecter la réponse
    print("Utilisateur : Comment résoudre l'équation  $2x + 5 = 13$  ?\n")
    print("Agent : ", end="")

    async for event in runner.run_async(
        user_id=USER_ID,
        session_id=SESSION_ID,
        new_message=user_message
    ):
        # Imprimer la réponse finale
        if event.is_final_response() and event.content and event.content.parts:
            print(event.content.parts[0].text)

# Étape 7 : Exécuter l'agent
# Pour Jupyter/Colab : utiliser await directement
# await run_agent()

# Pour les scripts Python : utiliser asyncio.run()
asyncio.run(run_agent())

```

Exécuter cet exemple

Dans un notebook Jupyter ou Google Colab :

```

# Utiliser simplement await (la boucle d'événements est déjà en cours d'exécution)
await run_agent()

```

Dans un script Python :

```

# Utiliser asyncio.run() pour démarrer la boucle d'événements

```

```
asyncio.run(run_agent())
```

Résultat attendu :

Session créée : session_001

Utilisateur : Comment puis-je résoudre l'équation $2x + 5 = 13$?

Agent : Excellente question ! Essayons de trouver la solution à ce problème ensemble.
Tout d'abord, selon vous,
que devons-nous faire pour isoler l'inconnue x d'un côté de l'équation ?

Quand utiliser l'exécution programmatique

Convient aux cas d'utilisation suivants :

-  Notebooks Jupyter et Google Colab
-  Applications Python personnalisées
-  Pipelines de traitement des données
-  Recherche et expérimentation
-  Contrôle précis de l'exécution

Pas idéal pour les cas d'utilisation suivants :

-  Tests interactifs rapides (utiliser `adk web` ou `adk run`)
-  Environnements autres que Python

Comparaison : quand utiliser chaque méthode

Méthode	Application idéale	Persistance de session	Complexité de configuration
<code>adk web</code>	Développement, débogage, démos	 Oui (dans un navigateur)	Faible
<code>adk run</code>	Tests rapides, workflows CLI	 Non	Très faible
<code>adk api_server</code>	Intégration d'API, production	Dépend du client	Faible
Programmatique	Applications personnalisées, notebooks	 Oui (gérée)	Moyenne

Résumé :

- Pour le développement ? Utilisez `adk web`
- Pour des tests rapides ? Utilisez `adk run`

- Pour créer une API ? Utilisez `adk api_server`
- Pour l'intégration personnalisée ? Utilisez l'exécution programmatique

Points à retenir

Choisissez le bon outil :

- Utilisez `adk web` pour le développement et le débogage visuels.
- Utilisez `adk run` pour effectuer des tests rapides en ligne de commande.
- Utilisez `adk api_server` pour le déploiement en tant que d'API.
- Utilisez l'exécution programmatique pour les applications Python personnalisées.

Toutes les méthodes fonctionnent avec le même agent :

- Votre fichier `agent.py` ne change pas.
- Les différentes méthodes sont simplement des façons différentes d'interagir.
- Choisissez celle qui convient à vos besoins actuels en termes de workflow.